Commande par RNA d'un système thermique à distance

Présentation de mi-parcours

Projet de Fin d'Année 2

Étudiants : Ouri Mohamed Amine et Chaanbi Hassen

Encadrant : M.Faouzi Bouani

Année universitaire :2025/2026

Sommaire

Introduction

Description du système thermique

La commande PID

La commande neuronale

Signaux d'étude

Conclusion

1. Introduction

Les systèmes thermiques présentent des comportements non linéaires notamment dues aux :

- Non linéarité dans les bus d'alimentation
- Propriétés physiques variables
- Échanges thermiques non linéaires

$\Longrightarrow$ commande précise difficile avec les méthodes classiques(Implémentation PID.
Question de commandabilité :
" Existe-t-il une commande u(·) qui améne le systéme, initialement de l'état x(0), à un état $x(t)$ quelconque au temps t = T ?"

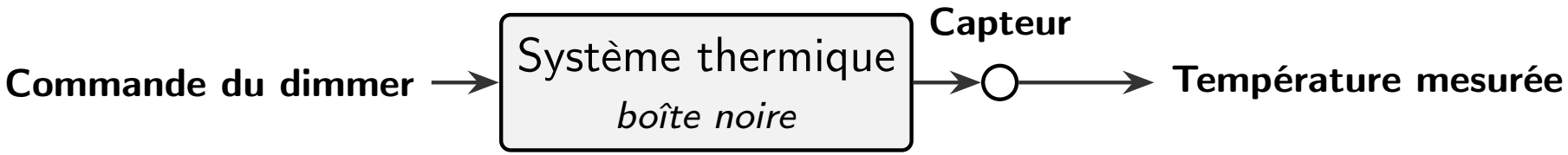
Problématique : Comment assurer une régulation efficace de la température d'un système thermique en utilisant une commande par réseaux de neurones artificiels ?

Objectifs du projet

1. Comprendre l'approche de la commande par résaux de neurones artificiel
2. Identification de la dynamique du système thermique à commander
3. Conception d'une commande par réseaux de neurones adapté au systéme
4. Validation par simulation
5. Essais pratiques et validation

2. Description du système thermique

Le système étudié dans ce projet est une boîte fermée, à l'intérieur de laquelle on souhaite contrôler la température.



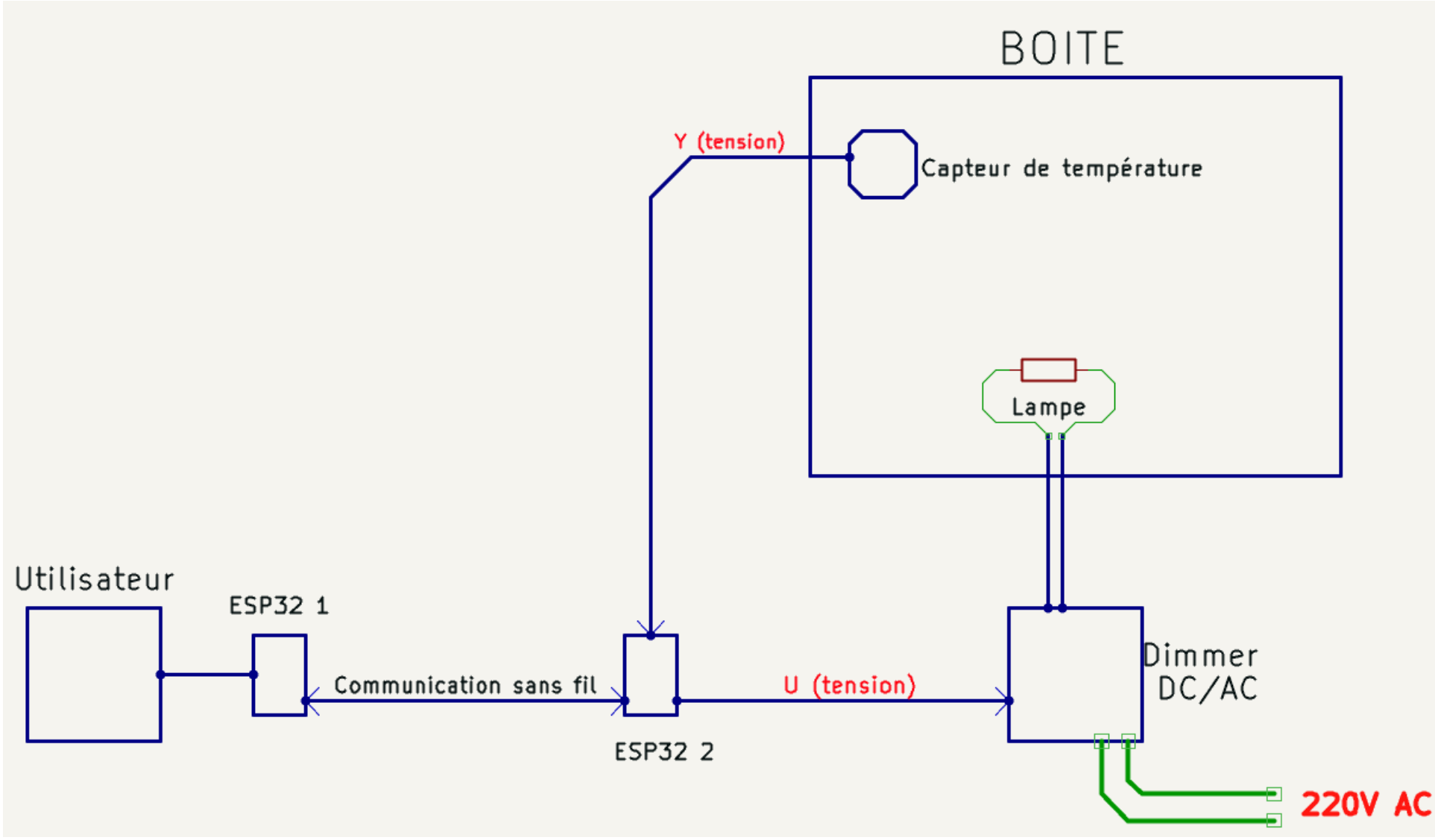


Figure – Schéma générale du systéme

3. La commande PID

Régulateur PID :Proportionel,Intégral,Dérivé

Systéme de premier ordre :
La fonction de transfert continue :

$$H(p) = \frac{K_s}{T_0.p + 1}$$

Gain statique : $K_s = 5$,Constante de temps $T_0 = 4$.
La fonction de transfert du système discret :

$$H(z) = \frac{0.4758}{z - 0.9048}$$

Fonction de transfert du régulateur PID :

$$C(z) = \frac{U(z)}{E(z)} = \frac{c_0 + c_1 z^{-1} + c_2 z^{-2}}{1 - z^{-1}}$$

Equation d'action du correcteur sur la dynamique de l'erreur :

$$u(k) = u(k-1) + c_0 \cdot e(k) + c_1 \cdot e(k-1) + c_2 . e(k-2)$$

Equation d'action du correcteur sous la forme continue :

$$u(t) = K_p\, e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{de(t)}{dt}$$

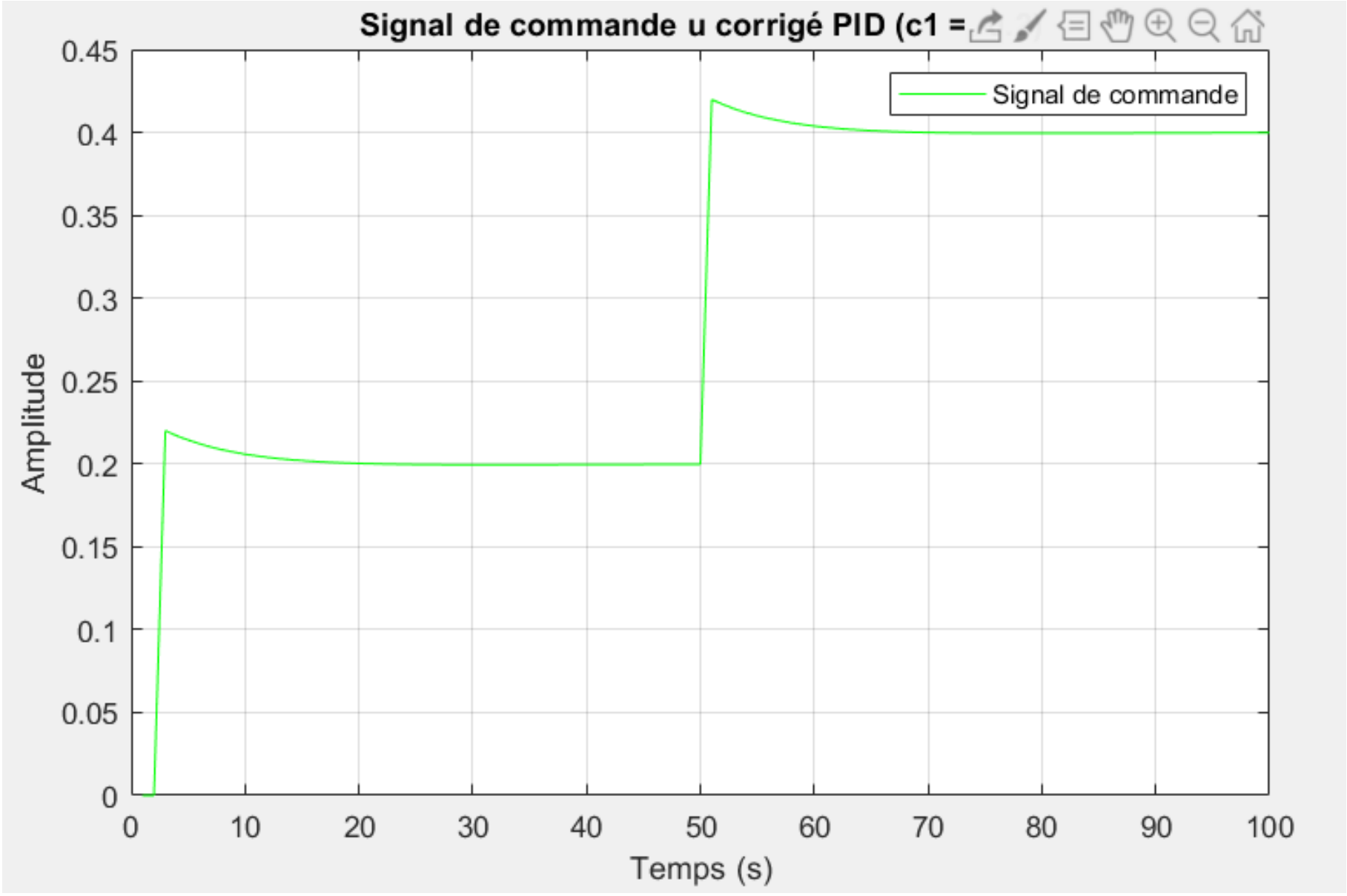
avec :

$$e(k) = y_{ref}(k) - y(k)$$

Régulation PID :

```
c0=0.22;
c1 =-0.20;%IL FAUT SAVOIR un tel c faire quoi
c2=0.00001;
%% --------------------La TZ de H par C2d utilisant par défaut'zoh'
Te=T0/10;
N = 100;          % nombre d'échantillons
Hz=c2d(H,Te,'zoh');
[b,a]=tfdata(Hz,'v');
%% -----------------Initialisation-------------------------
y_ref=[];
for k=1:N/2
   y_ref(k)=1;
end
for k=N/2+1:N
   y_ref(k)=2;
end
y(1) =0 ;y(2)=0
e(2)=0;e(1)=0;
u(2)=0;u(1)=0;
% --- Simulation de la réponse par équation récurrente ---
for k = 3:N
    %y(k) = -a1*y(k-1) - a2*y(k-2) + b0*y_ref(k) + b1*y_ref(k-1);
    y(k)=-a(2)*y(k-1)+b(2)*u(k-1);
    e(k)=y_ref(k)-y(k);
    u(k)=u(k-1)+c0*e(k)+c1*e(k-1)+c2*e(k-2);
end
```

Commande corrigée :



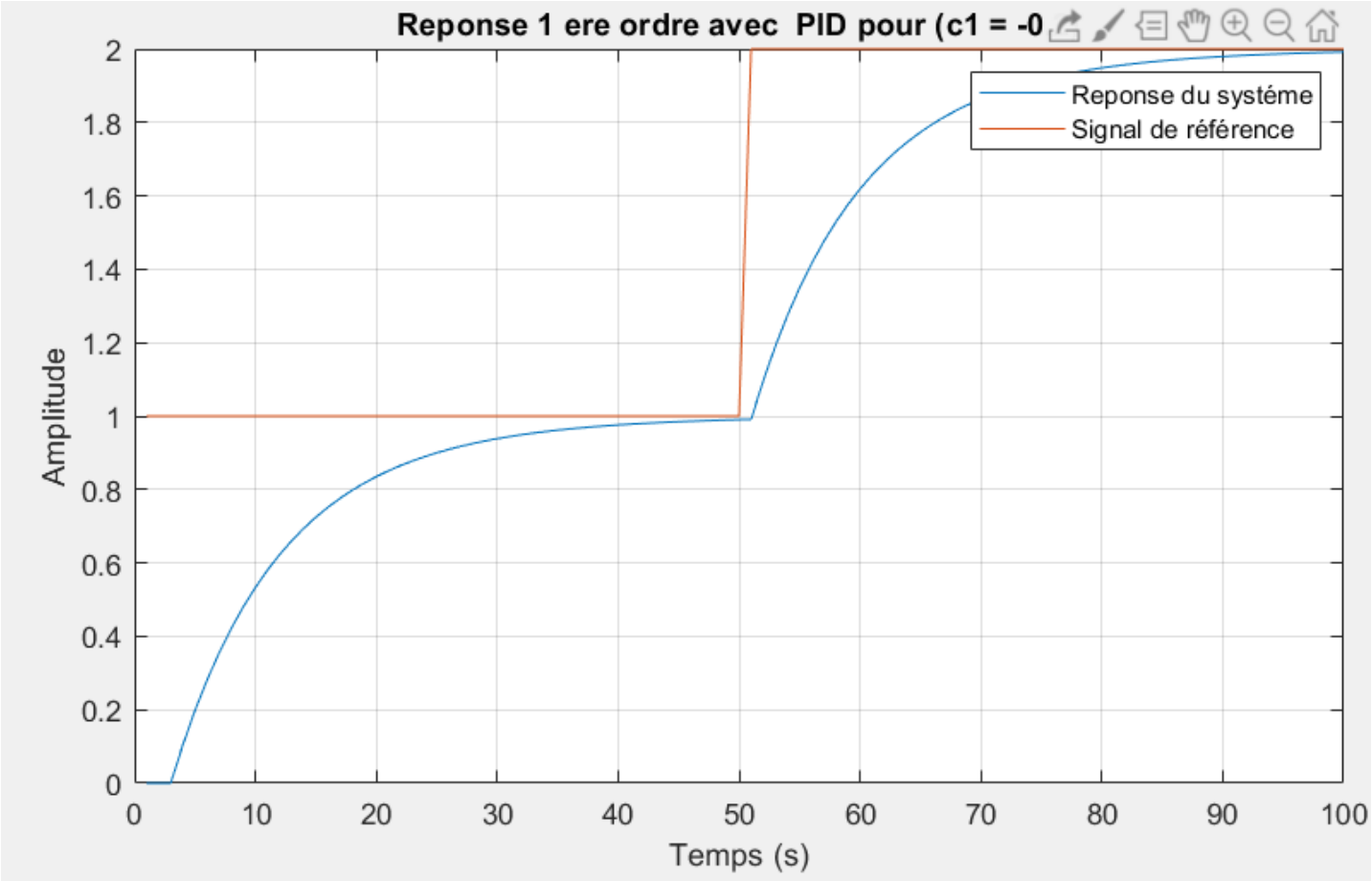


Figure – Sortie systéme et consigne

Systéme de deuxiéme ordre

Fonction de transfert en p :

$$H(p) = \frac{Ks\omega_0^2}{p^2 + 2\xi\omega_0 p + \omega_0^2}$$

Avec :

$$Ks = 5, \quad \omega_0 = 20, \quad \xi = 0.5$$

On obtient :

$$H(p) = \frac{2000}{p^2 + 20p + 400}$$

Fonction de transfert en z obtenue par la méthode de bloqueur d'ordre 0 "zoh" :

$$H(z) = \frac{0.01217\, z^{-1} - 0.0003412\, z^{-2}}{1 - 0.05459\, z^{-1} + 0.0007466\, z^{-2}}$$

Régulation PID :

```
clc;clear;close all;
%% Comment Utuliser ce script %%


%CE PROGRAMME ILLUSTRE L'APPLICATION DE LA COMMANDE PID SUR UN SYSTEME DE
%DEUXIEME ORDRE DE PARAMETRES(Ks,xi,w0),Ks ETANT LE GAIN,xi LE COEFFICIENT
%D'AMMORTISSEMENT ET w0 LA PULSATION PROPRE DE SYSTEME




%% ==================%%
%% Les paramétres du correcteur
c0=15;%Kp
c1 =1; %Ki
c2=1;%Kd
%% ----------------------Reponse en continue--------------------------
Ks=5;
w0=20;
xi=0.5;
Num_p=[Ks];
Den_p=[1 2*xi*w0 w0^2];
H=tf(Num_p,Den_p);
step(H);
%% --------------------La TZ de H par C2d utilisant par défaut'zoh'
Te=0.36;
N = 100;          % nombre d'échantillons
Hz=c2d(H,Te,'zoh');
[b,a]=tfdata(Hz,'v');
```

Correction de la commande :

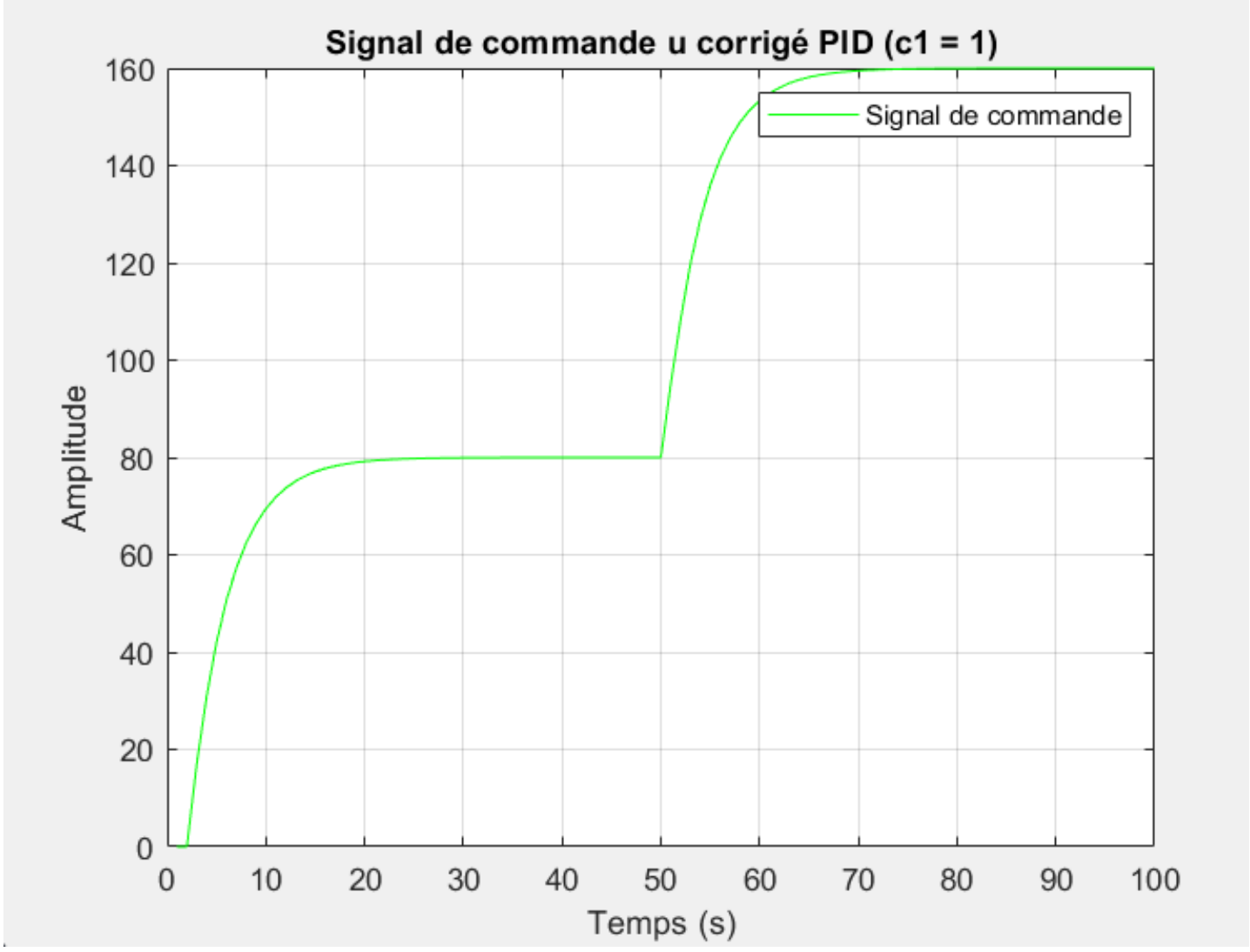


Figure – Commande systéme deuxiéme ordre corrigée par PID

Sortie systéme deuxiéme ordre au processus PID :

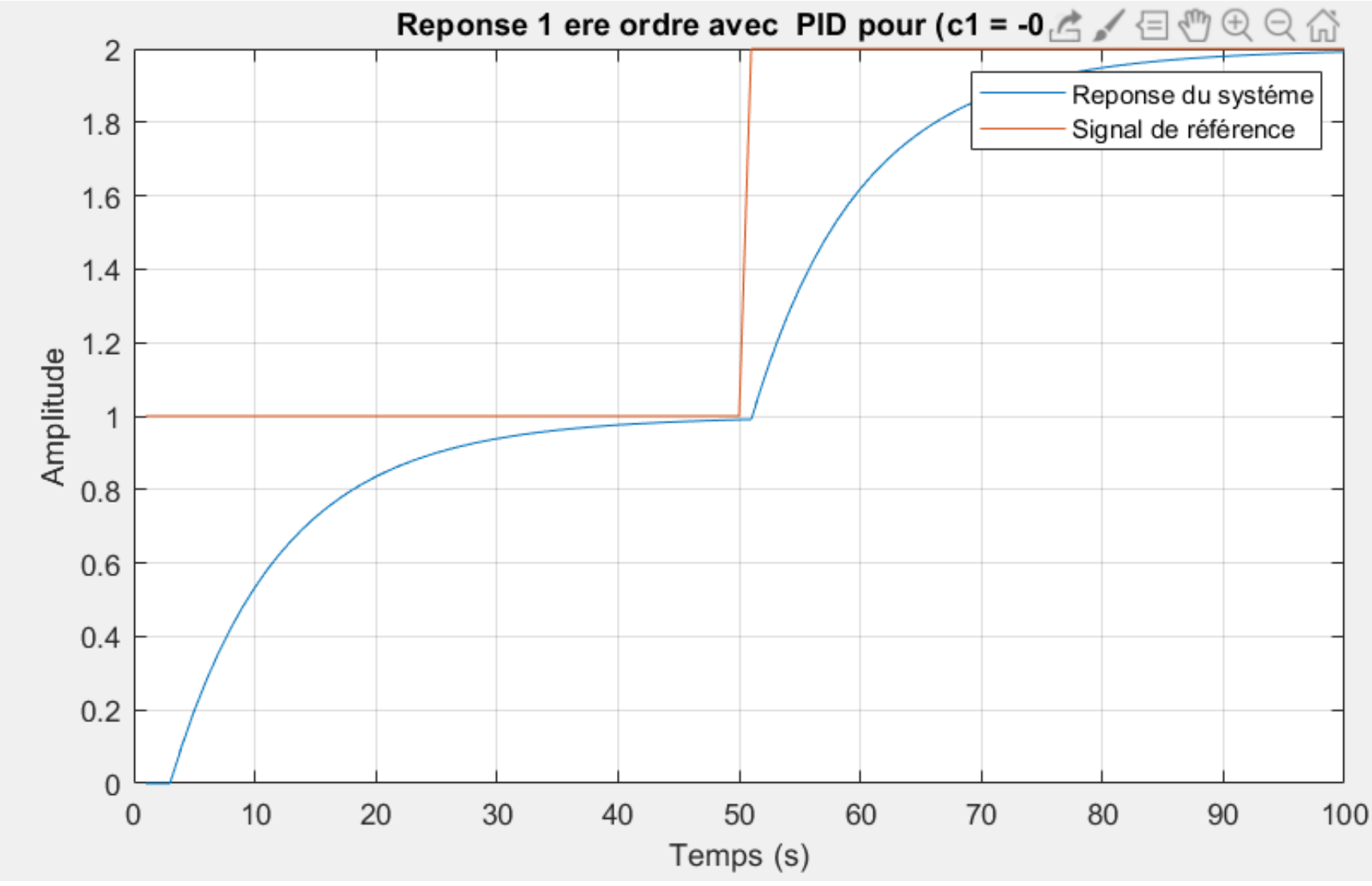


Figure – Réponse systéme deuxiéme ordre régulé par PID

4. La commande neuronale

Un neurone biologique : Le potentiel de stimulation doit passer le seuil -55 mV pour activer le neurone

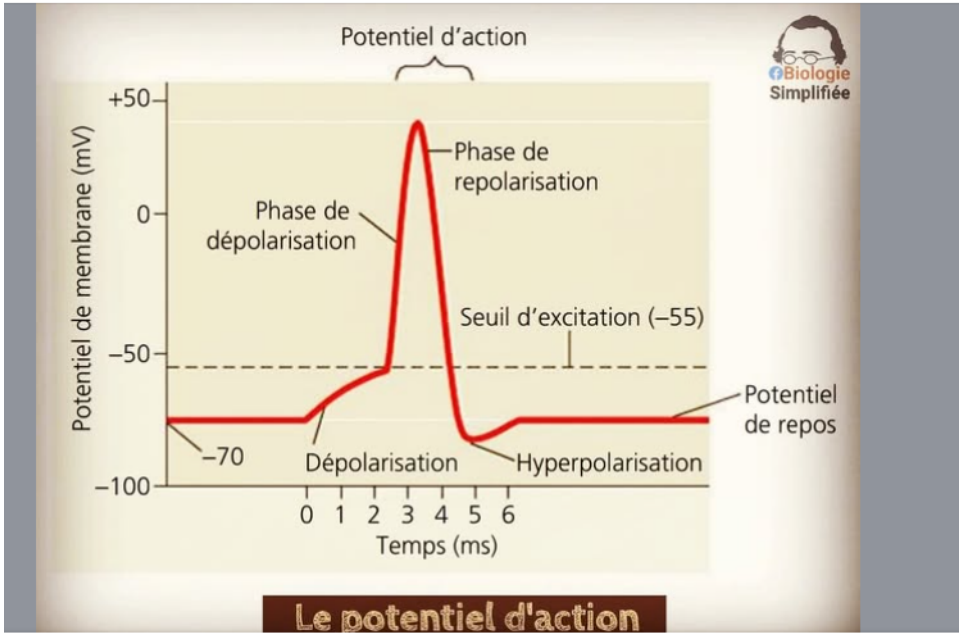


Figure – Seuil d'activation

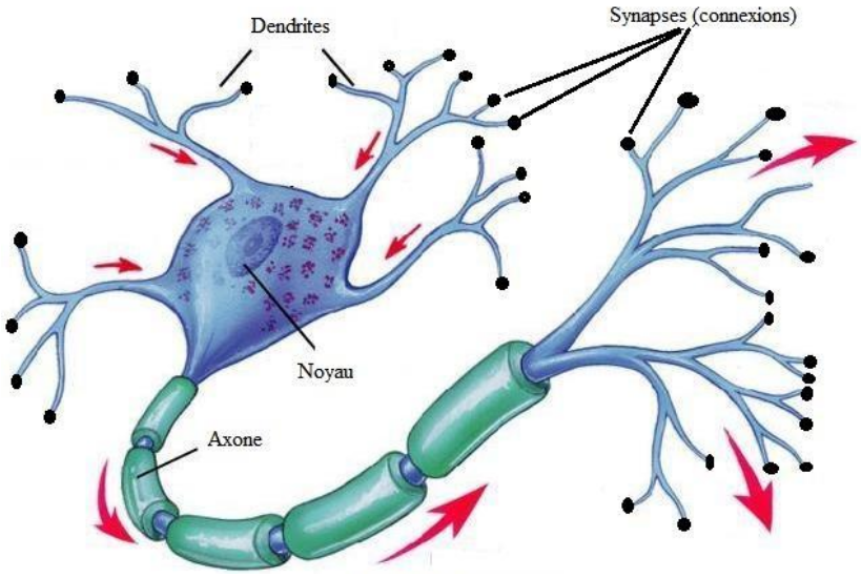


Figure – Neurone biologique

Un neurone artificiel : Fonction d'activation la plus proche du cas bilogique :

$$f(u) = \begin{cases} 1 & \text{si } \sum w_i x_i - \theta \geq 0 \\ 0 & \text{sinon} \end{cases}$$

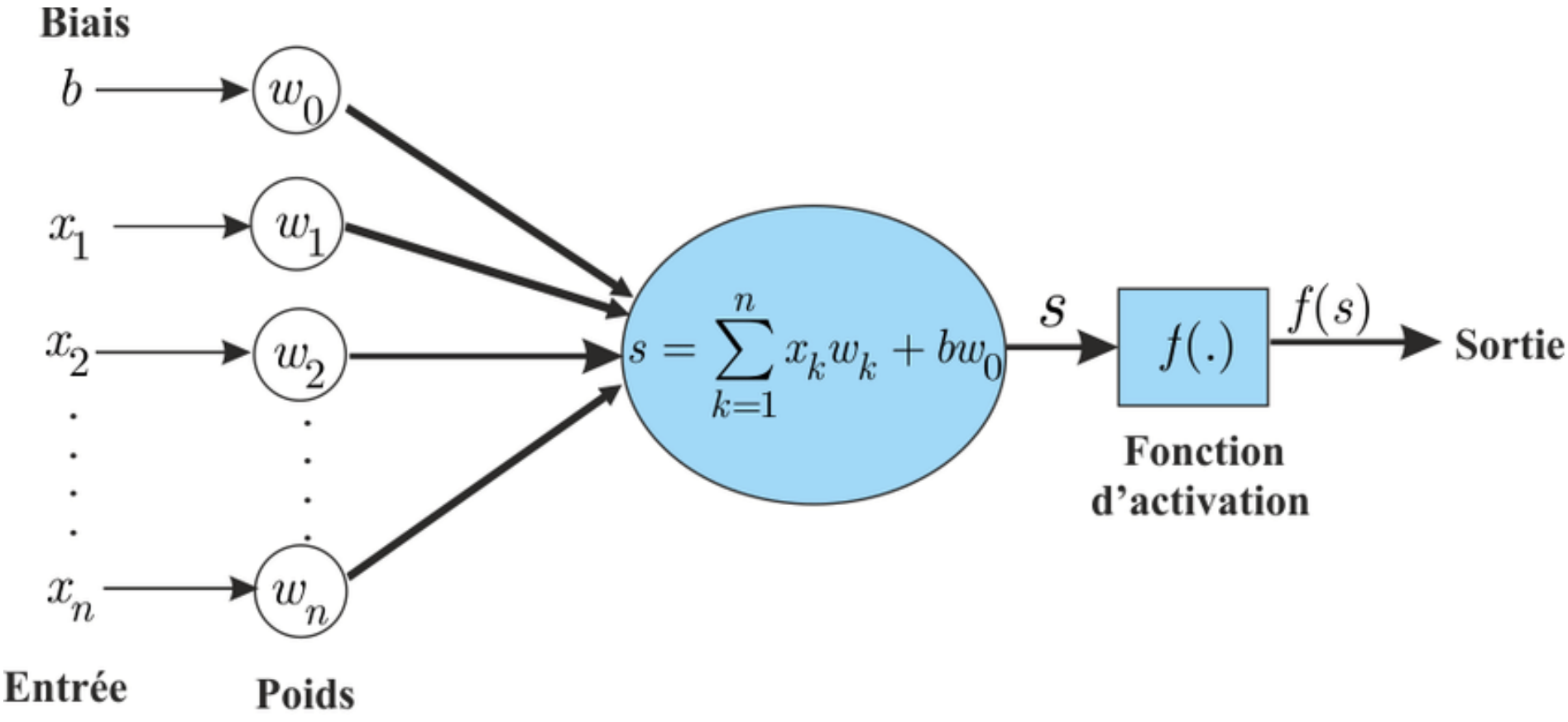


Figure – modéle du neurone artificiel

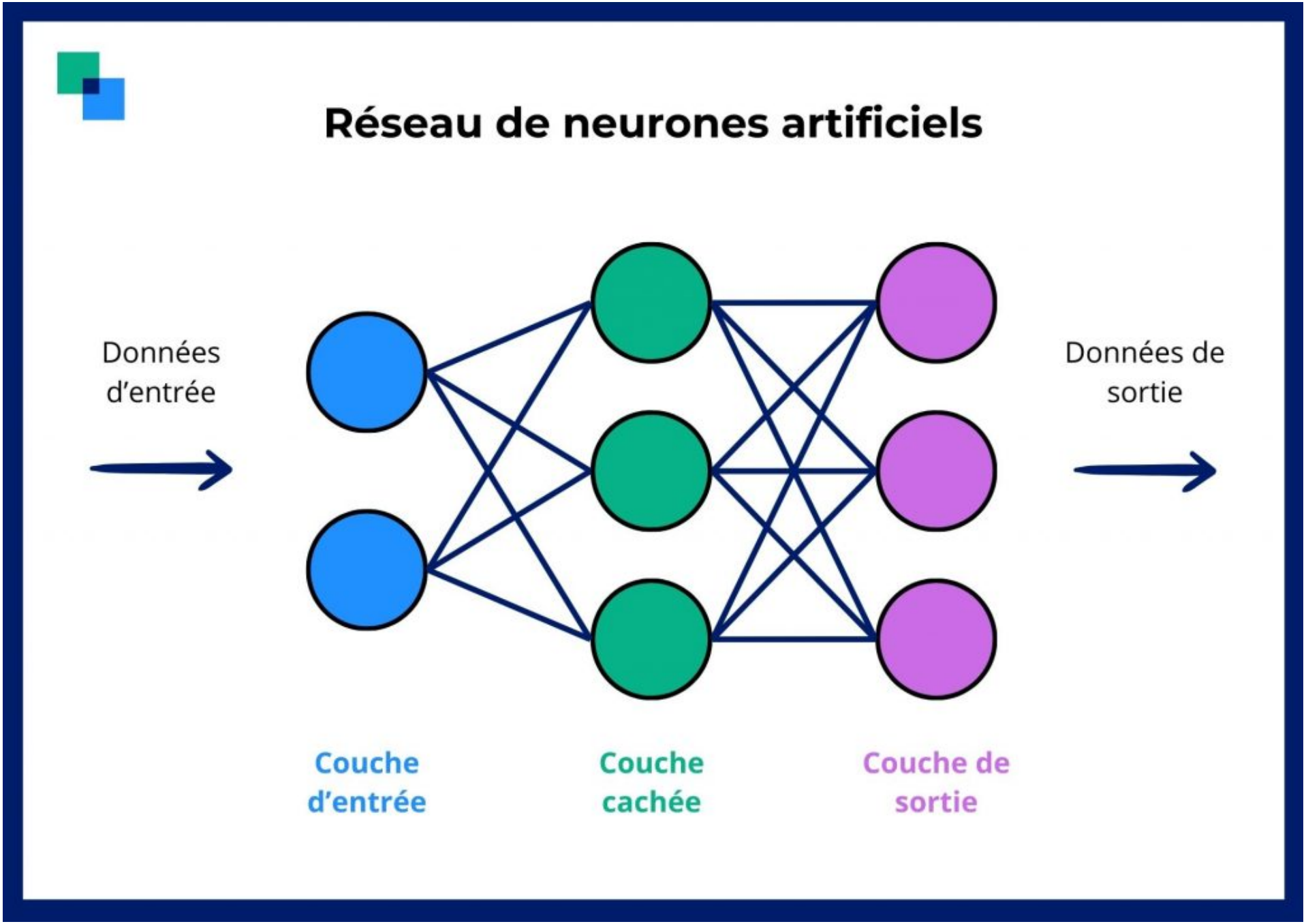


Figure – réseau de neurones

a. Le Neurone : Un Système Non Linéaire

Calcul d'un neurone :

$$y = f(w \cdot x - b) \tag{1}$$

où :

- x : entrée
- w : poids (paramètre à apprendre)
- b : biais (paramètre à apprendre)
- f : fonction d'activation du neurone (ex : sigmoid,tanh..)
- y : sortie du neurone

La non-linéarité f permet d'apprendre des relations complexes !
Remarque : $w \cdot x - b$? En absence du biais le neurone implémenté devient plus sensible au petites variations de ces entrées $\Longrightarrow$b toujours strictement négatif.

b. La Mémoire : Les Poids et Biais

La mémoire du réseau = ses paramètres :

$$\text{Mémoire} = \{w_1, w_2, \ldots, w_n, b_1, b_2, \ldots, b_m\} \tag{2}$$

L'erreur mesure la qualité de la mémoire :

$$\text{Erreur} = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2 \tag{3}$$

où :

- y_i : valeur attendue
- $\hat{y}_i$: valeur prédite par le réseau
- N : nombre d'exemples

c. Comment Améliorer ? Calculer le Gradient

Le gradient indique comment modifier les poids :

$$\text{Gradient} = \frac{\partial \text{Erreur}}{\partial w} \tag{4}$$

Ce gradient nous dit :

- Si le gradient est **positif** : diminuer w
- Si le gradient est **négatif** : augmenter w

Rétropropagation : méthode efficace pour calculer tous les gradients en remontant le réseau couche par couche.

d. Mise à Jour de la Mémoire

Règle simple de mise à jour :

$$w_{\text{new}} = w_{\text{old}} - \alpha \times \text{Gradient}, b_{\text{new}} = b_{\text{old}} - \alpha \times \text{Gradient} \tag{5}$$

où α = taux d'apprentissage (Pas d'adaptation)

Résumé du processus d'apprentissage :

1. Calculer la sortie du réseaux en fonction des paramétres du réseaux initialisées
2. Calculer l'erreur quadratique sur N échantillons
3. Calculer les gradients(Voit la dynamique de l'erreur en fonction des poids)
4. Mettre à jour les poids
5. Répéter la boucle d'apprentissage pour plusieurs passage(Nombre de passage) jusqu'à approcher de la sortie systéme étudié

La capacité du réseaux en mémoire à suivre la sortie (TARGET) s'améliore pour un nombre de passage trés grand !

4. Signaux d'étude

Signal déterministe sans mémoire

$$g(k) = 0.4\sin(0.2k) + 0.6\cos(0.4k)$$

- Signal borné et oscillant.
- Permet d'exciter la dynamique du système.

Signal déterministe récurcif :
Le second signal est issu d'un système non linéaire défini par :

$$\begin{cases} y(k) = \dfrac{-0.8\, y(k-2)\, u^2(k-1)}{1 + u^2(k-1)} + 0.033\, y(k-1) + 0.5\, u(k-1) \end{cases}$$

- Dynamique non linéaire .
- Présence de retards temporels.

Architectures de réseaux de neurones

Deux architectures de réseaux de neurones ont été étudiées :

- Réseau à **1 neurone**
- Réseau à **10 neurones**

Dans les deux architectures du réseaux on implémente la boucle d'apprentissage par la fonction d'activation :

$$f(v) = \tanh(v) = \frac{e^{v} - e^{-v}}{e^{v} + e^{-v}}$$

Caractéristiques :

- Fonction **non linéaire**, indispensable pour l'apprentissage.
- Fonction **symétrique** par rapport à l'origine.
- Sortie bornée dans l'intervalle $[-1, 1]$.
- Meilleure convergence que la sigmoïde grâce au centrage autour de zéro.
- Largement utilisée dans les couches cachées des RNA.

Phase d'apprentissage

Algorithme d'apprentissage :

```
%% APPRENTISSAGE
for p = 1:pmax
    for k = 3:N
        v = -w0 + w1 * g(k-1) + w2 * g(k-2);
        y(k) = tanh(v); % sortie du neurone

        %  Termes de correction et de retropropagation( Backpropagation )
        df =  1 - tanh(v)^2;

        dw0 = mu * (g(k) - y(k)) * df * (-1);
        dw1 = mu * (g(k) - y(k)) * df * g(k-1);
        dw2 = mu * (g(k) - y(k)) * df * g(k-2);
        w0 = w0 + dw0;
        w1 = w1 + dw1;
        w2 = w2 + dw2;
    end

    % critère : somme des erreurs au carré
    Cr(p) = (g - y) * (g - y)';
end
```

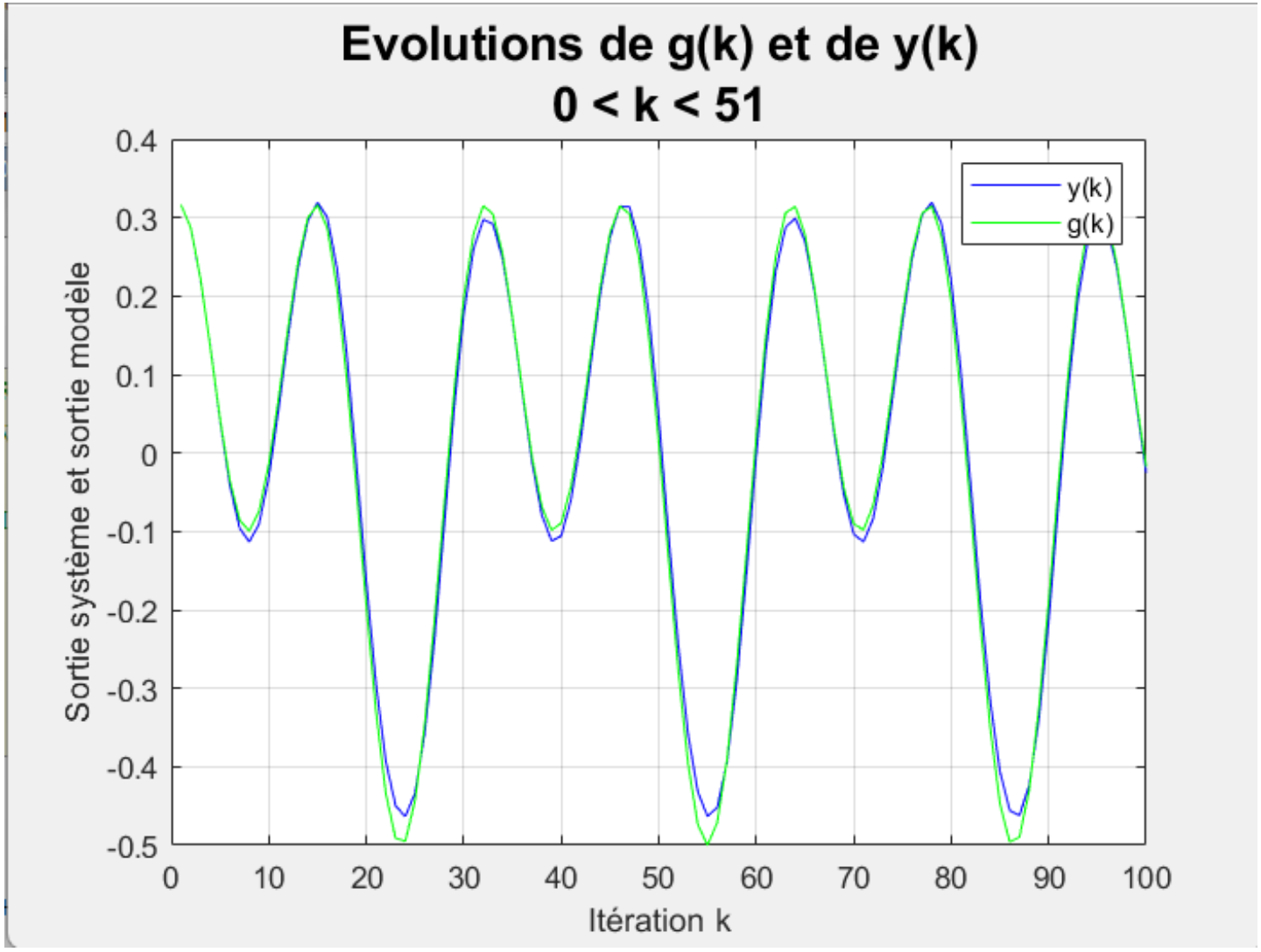


Figure – Sortie du systéme et du RNA

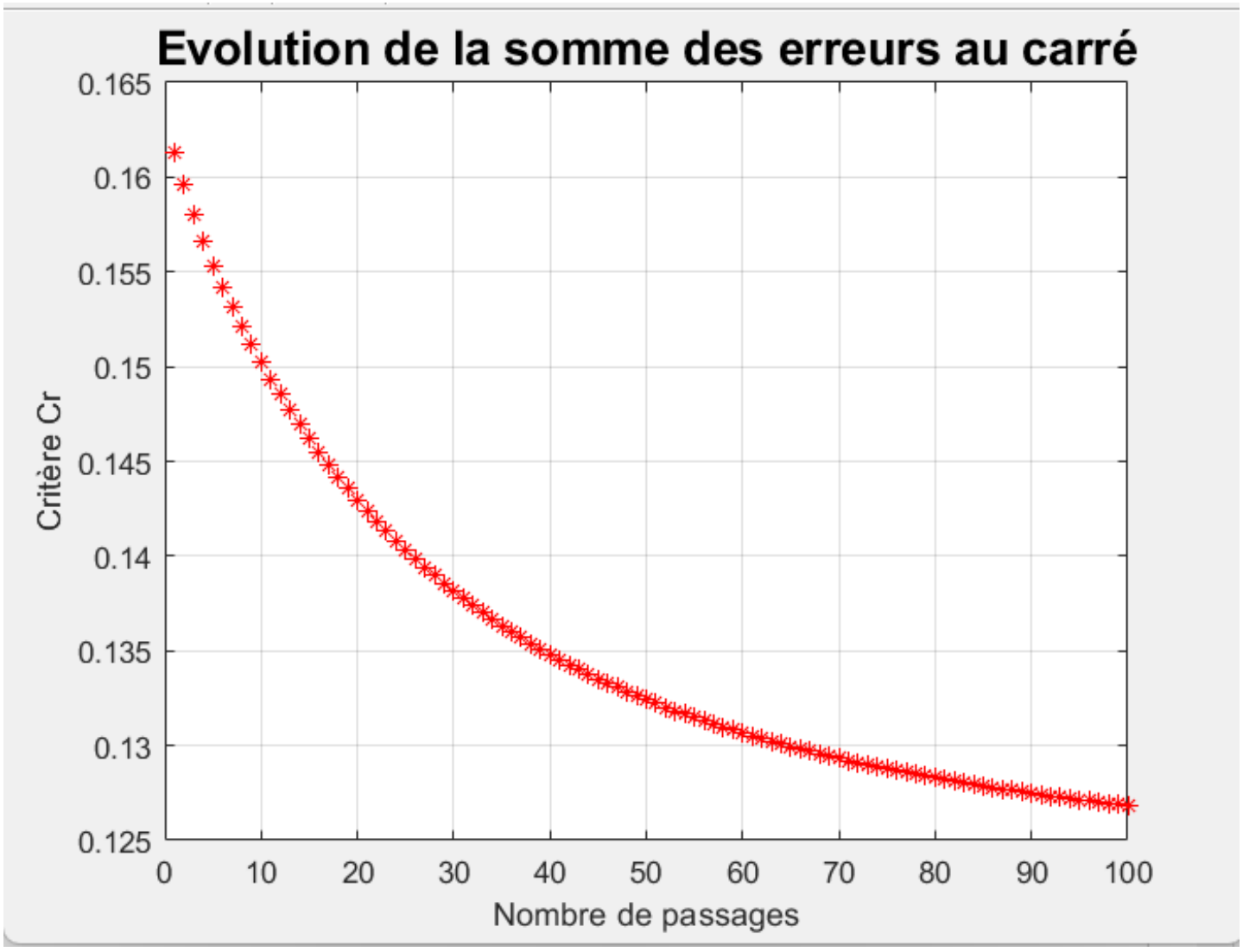


Figure – Evolution de l'erreur quadratique en fonction du nombre de passage

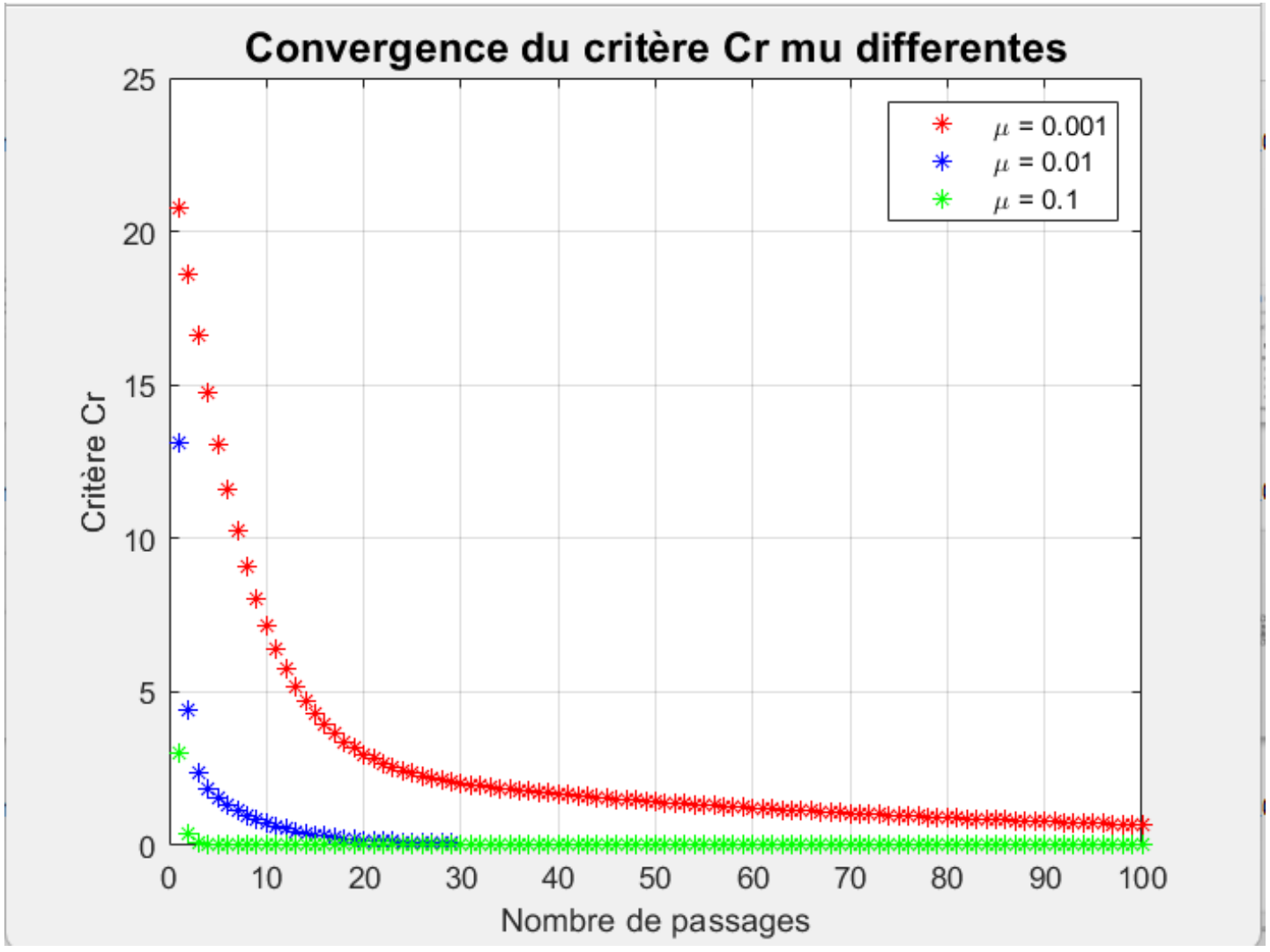


Figure – Evolution du critére par balayage du pas d'adaptation μ

Signal récurcif

Signal avec mémoire y(k) :

$$u = rand(1, N) - 0.5$$

$$y(1) = 0, \quad y(2) = 0$$

$$y_1 = \mathbf{0}_{1\times N}$$

Réseaux de 1 neurones

Code matlab :

```
%% LIRE CE COMMENTAIRE ICI POUR COMPRENDRE DE QUOI S'AGIT LE SCRIPT
%VISUALISATION DU CRITERE "Cr" EN FONCTION DU PAS D'ADAPTATION "mu" POUR UN
%RESEAUX DE "1 NEURONE" pour 3 valeurs differentes
%CACHEE
```

```
%% initialisation
N = 100; % nombre de mesures
mu_v=[0.001 0.01 0.1];% vecteurs de valeurs mu à tester
%mu = 0.2; % pas d'adaptation
u = rand(1,N)-0.5;
y(1)=0;
y(2)=0;
pmax = 100; % nombre maximal de passages
Cr = zeros(3,pmax); % stockage des critères
for k = 3:N
    a= y(k-2);
    y(k) = -0.8*a*(u(k-1))^2/(1+(u(k-1))^2)+ 0.033*y(k-1) + 0.5*u(k-1);
end
```

```
%%  Initialisation des paramètres du neurone artificiel
pmax = 100; % nombre maximal de passages
```

```
%%  valeurs initiales de la sortie du neurone
y(1) =0;
y(2) = 0;
```

Suite :

```
%% APPRENTISSAGE
for i=1:3
    mu = mu_v(i);   % pas d'adaptation courante
    w0 = 0.5;
    w1 = -0.1;
    w2 = 0.1;
    y1 = zeros(1,N);
    for p = 1:pmax
        for k = 3:N
             v = -w0 + w1 * u(k-1) + w2 * y(k-1);
             y1(k) = tanh(v); % sortie du neurone

              %  Termes de correction et de retropropagation( Backpropagation )
             df =  1 - tanh(v)^2;

            dw0 = mu * (y(k) - y1(k)) * df * (-1);
            dw1 = mu * (y(k) - y1(k)) * df * u(k-1);
            dw2 = mu * (y(k) - y1(k)) * df * y(k-1);
             w0 = w0 + dw0;
             w1 = w1 + dw1;
             w2 = w2 + dw2;
    end

    % critère : somme des erreurs au carré
    Cr(i,p) = (y - y1) * (y - y1)';
end
end
```

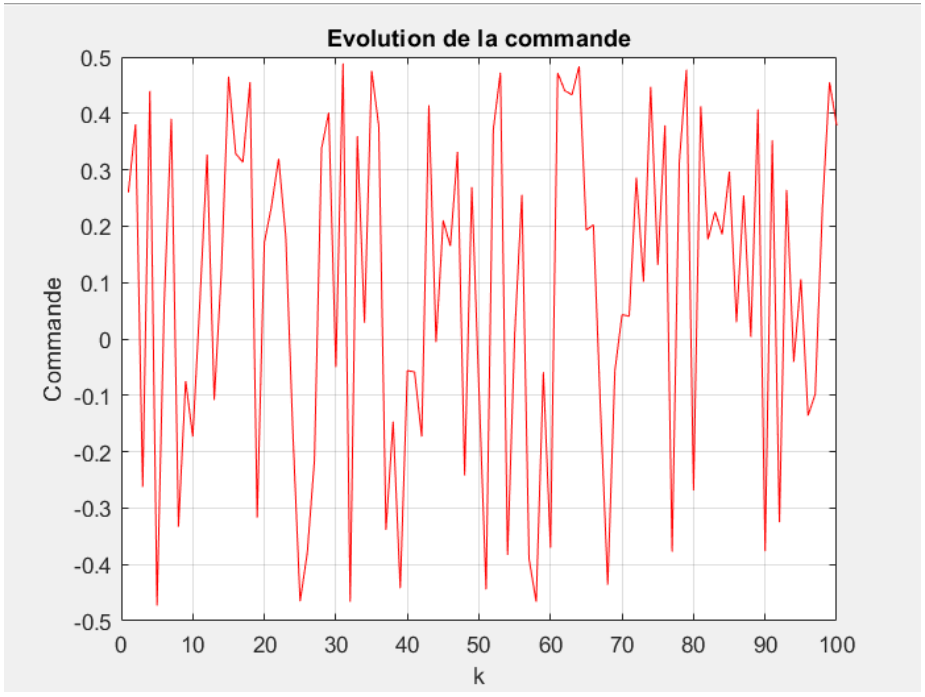


Figure – Signal de commande

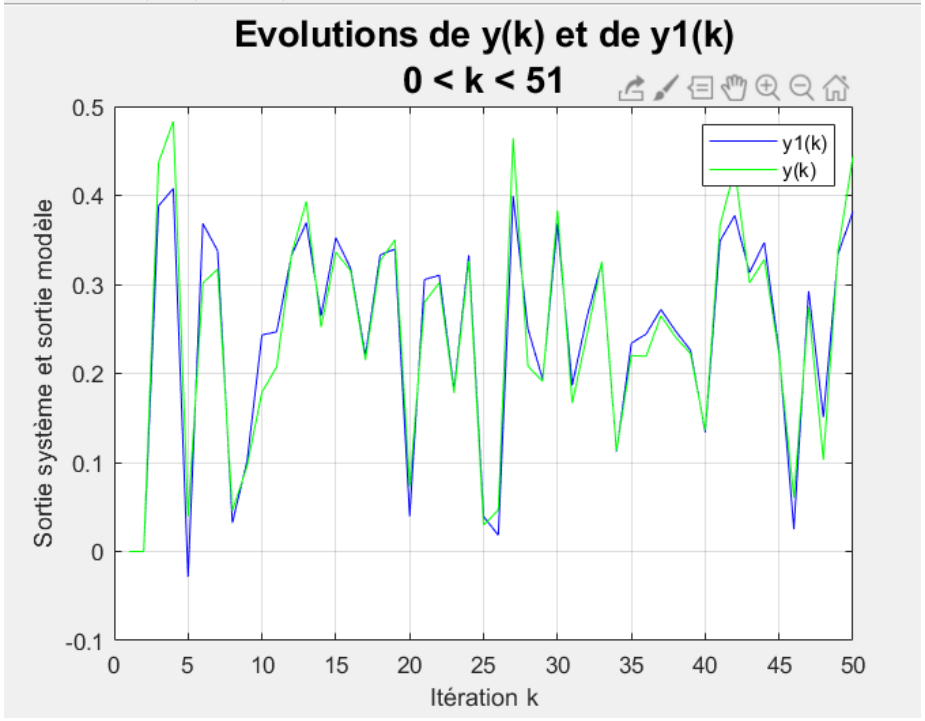
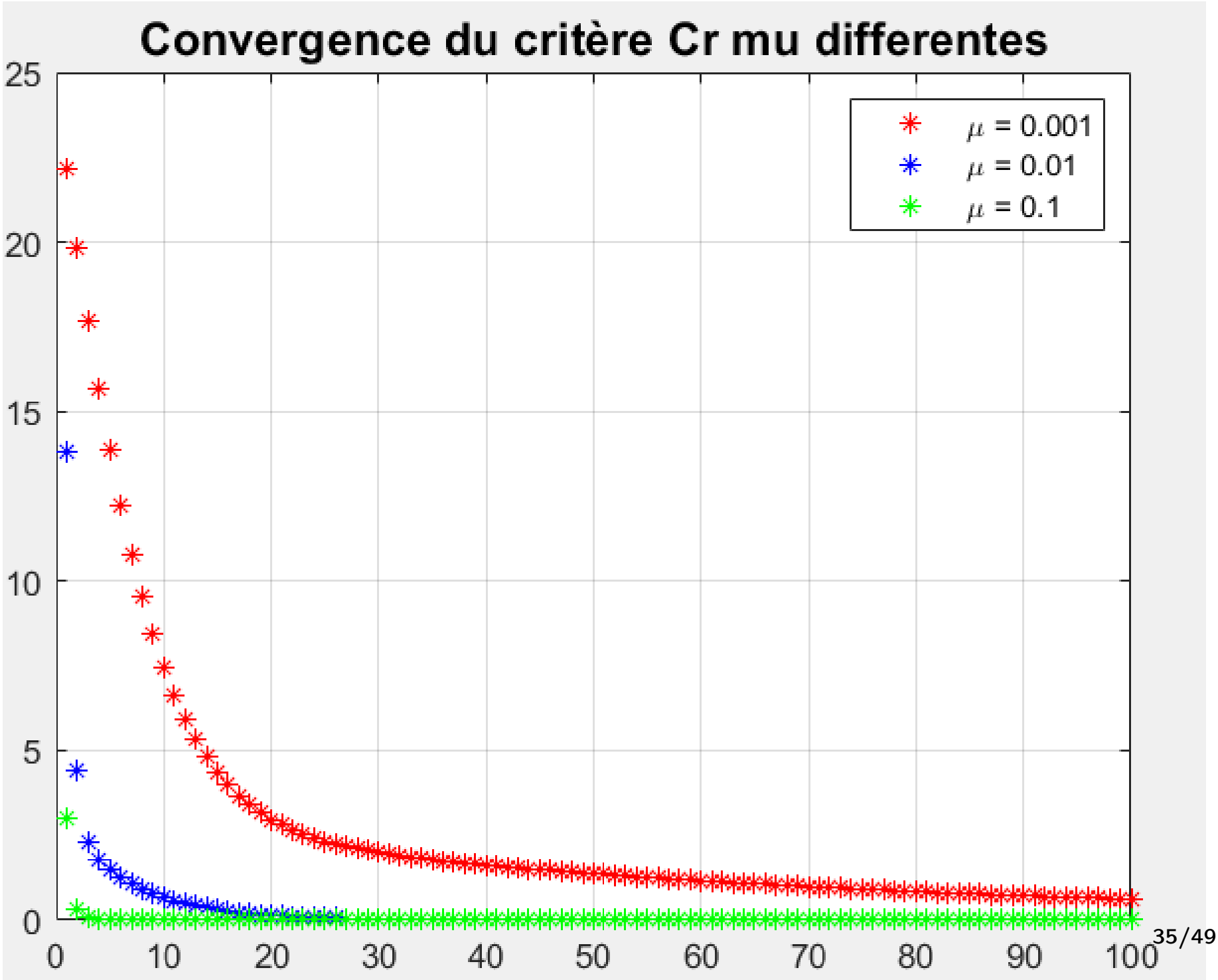


Figure – Sortie systéme et sortie RNA


Convergence du critère Cr mu differentes
25
20
15
10
5
0
0
10
20
30
40
50
60
70
80
90
100
μ = 0.001
μ = 0.01
μ = 0.1

Réseaux de 10 neurones

```
clear all
close all
clc
%% LIRE CE COMMENTAIRE ICI POUR COMPRENDRE DE QUOI S'AGIT LE SCRIPT
%VISUALISATION DU CRITERE "Cr" EN FONCTION DU PAS D'ADAPTATION "mu" POUR 3 VALEURS
%DIFFERENTES(0.001,0.01 et 0.1)POUR RESEAUX DE 10 NEURONES SUR COUCHES
%CACHEE

%% ---------------- INITIALISATION ----------------
mu_v = [0.001 0.01 0.1];    % 3 valeur de pas d'adaptation
pmax = 50;                   % nombre maximal de passages
N = 100;                    % nombre de mesures
n = 10;                      % nombre de neurones sur la couche cachee

Cr = zeros(3,pmax); % stockage des critères

%% ---------------- SIGNAUX D'ENTREE ----------------
u = rand(1,N)-0.5;% sera
%y(1) = zeros(1,N);
y(1) = 0;
y(2)=0;
y1 = zeros(1,N);
% REPONSE DU SYSTEME ----------------
for k = 3:N
    a = y(k-2);
    y(k) = -0.8*a*(u(k-1))^2/(1+(u(k-1))^2)+ 0.033*y(k-1) + 0.5*u(k-1);
end
```

Suite du code :

```
%% ================== APPRENTISSAGE ==================
% pour chaque valeur de mu  courante on initialise les poids et on calcul l'apprentissage et Cr
for i = 1:3
    mu = mu_v(i);   % pas d'adaptation courant
    % --- Initialisation des poids ---
    W1 = 0.1*randn(n,2);
    b1 = 0.1*rand(n,1);
    W2 = 0.1*randn(1,n);
    b2 = 0.1*randn;

    for p = 1:pmax   % Boucle passages

        for k = 3:N  % Boucle mesures

            % Vecteur d'entrée
            X = [y(k-1); u(k-1)];

            % Propagation avant
            V1 = W1*X - b1;
            S1 = tanh(V1);
            V2 = W2*S1 - b2;
            y1(k) = tanh(V2);

            % Erreur
            e = y(k) - y1(k);
```

Suite du code :

```
            %% Rétropropagation
            df2 = e*(1 - y1(k)^2);
            W2 = W2 + mu*df2*S1';
            b2 = b2 - mu*df2;

            df1 = (W2'*df2).*(1 - S1.^2);
            W1 = W1 + mu*df1*X';
            b1 = b1 - mu*df1;
        end

        % Critère quadratique moyen
        Cr(i,p) = (y - y1)*(y - y1)'/N;
    end
end
```

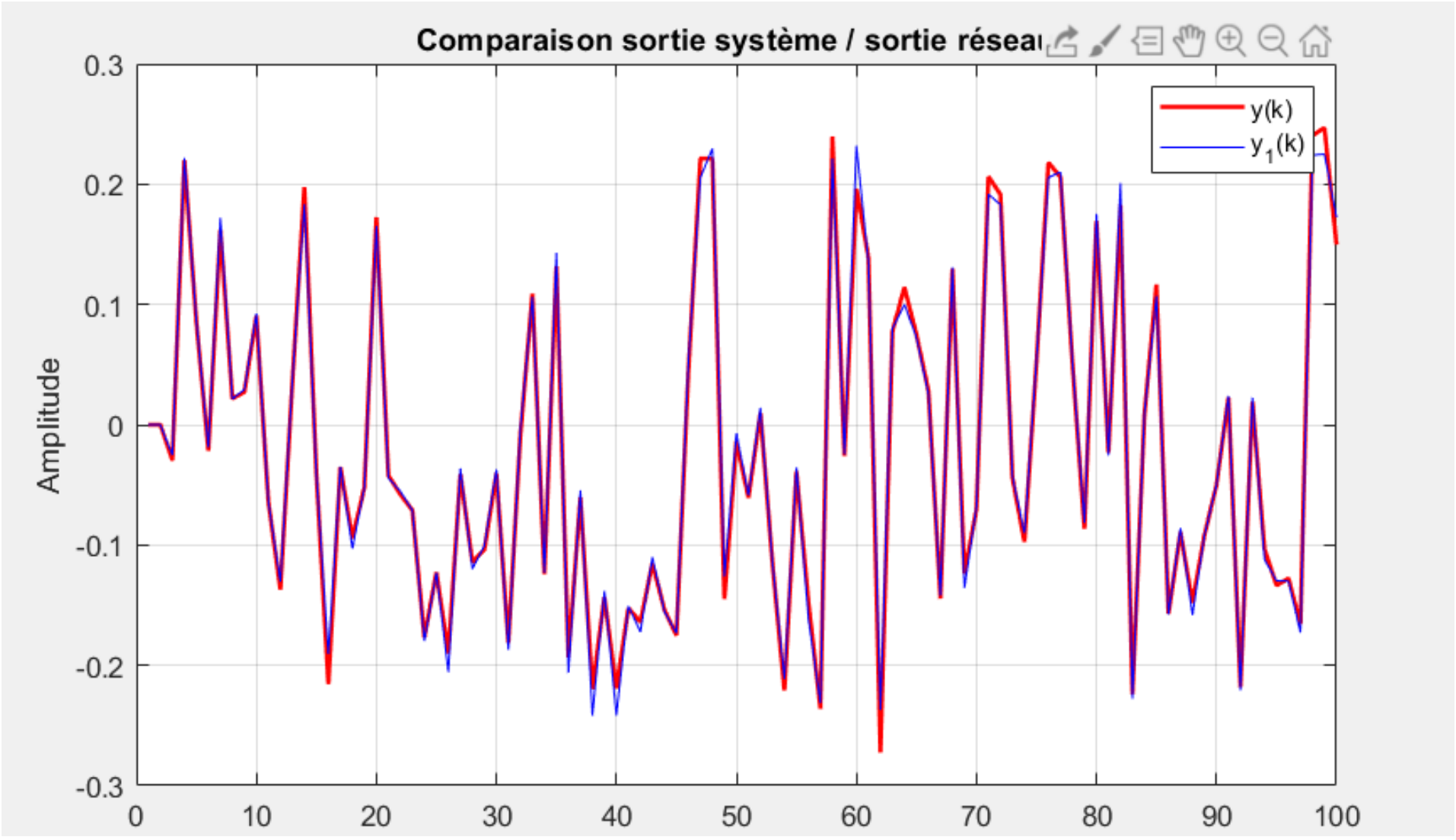


Figure – Sortie systéme et sortie RNA

Test

Réseaux de 1 neurone :

```
%% ================ TEST  ==================
Ntest = 100;
u_test =0.5*(rand(1,Ntest)-0.5);

y_test = zeros(1,Ntest);
y1_test = zeros(1,Ntest);

y_test(1)=0;
y_test(2)=0;
```

```
%% Réponse réelle du système (validation)
for k = 3:Ntest
    a_test = y_test(k-2);
    y_test(k) = -0.8*a_test*(u_test(k-1))^2/(1+(u_test(k-1))^2)+ 0.033*y_test(k-1) + 0.5*u_test(
end
```

```
%% Réponse du neurone (poids figés)
for k = 3:Ntest
    v_test = -w0 + w1*u_test(k-1) + w2*y_test(k-1);
    y1_test(k) = tanh(v_test);
end
```

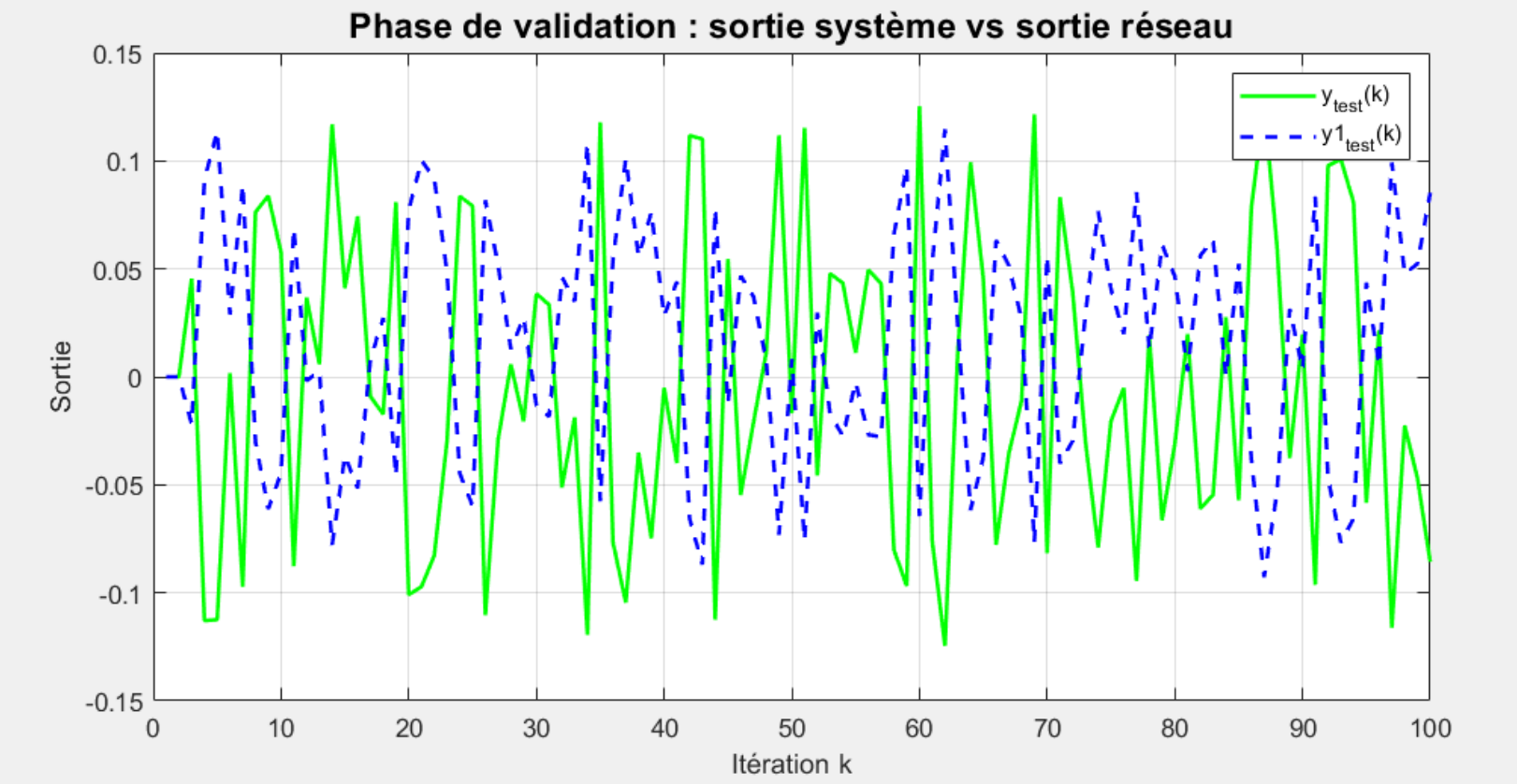


Figure – Evolution de la sortie du réseaux pendant le test

$\Longrightarrow$ capacité isuffisante pour reproduire la dynamique non linéaire du système.

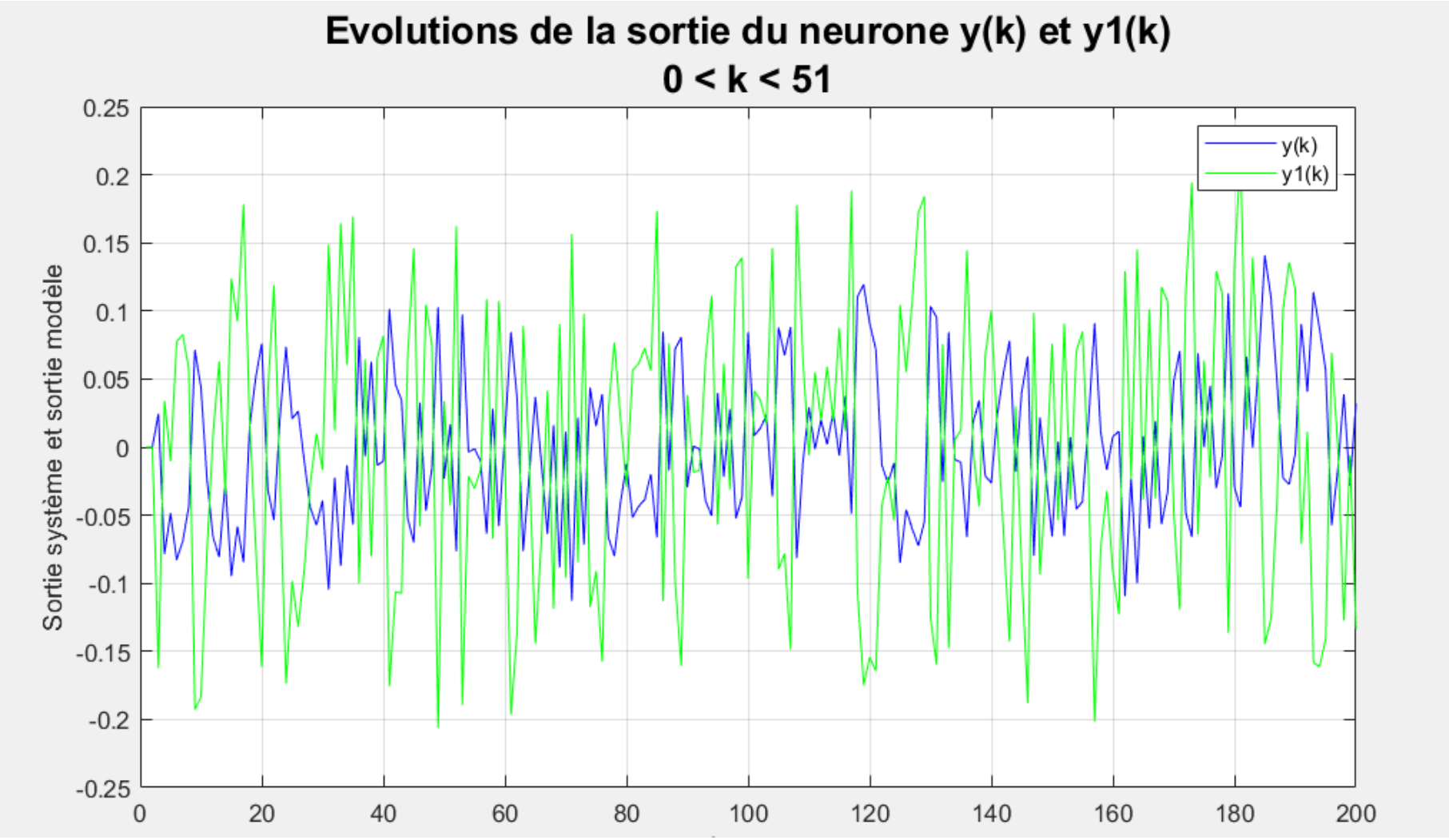


Figure – Phase d'apprentissage

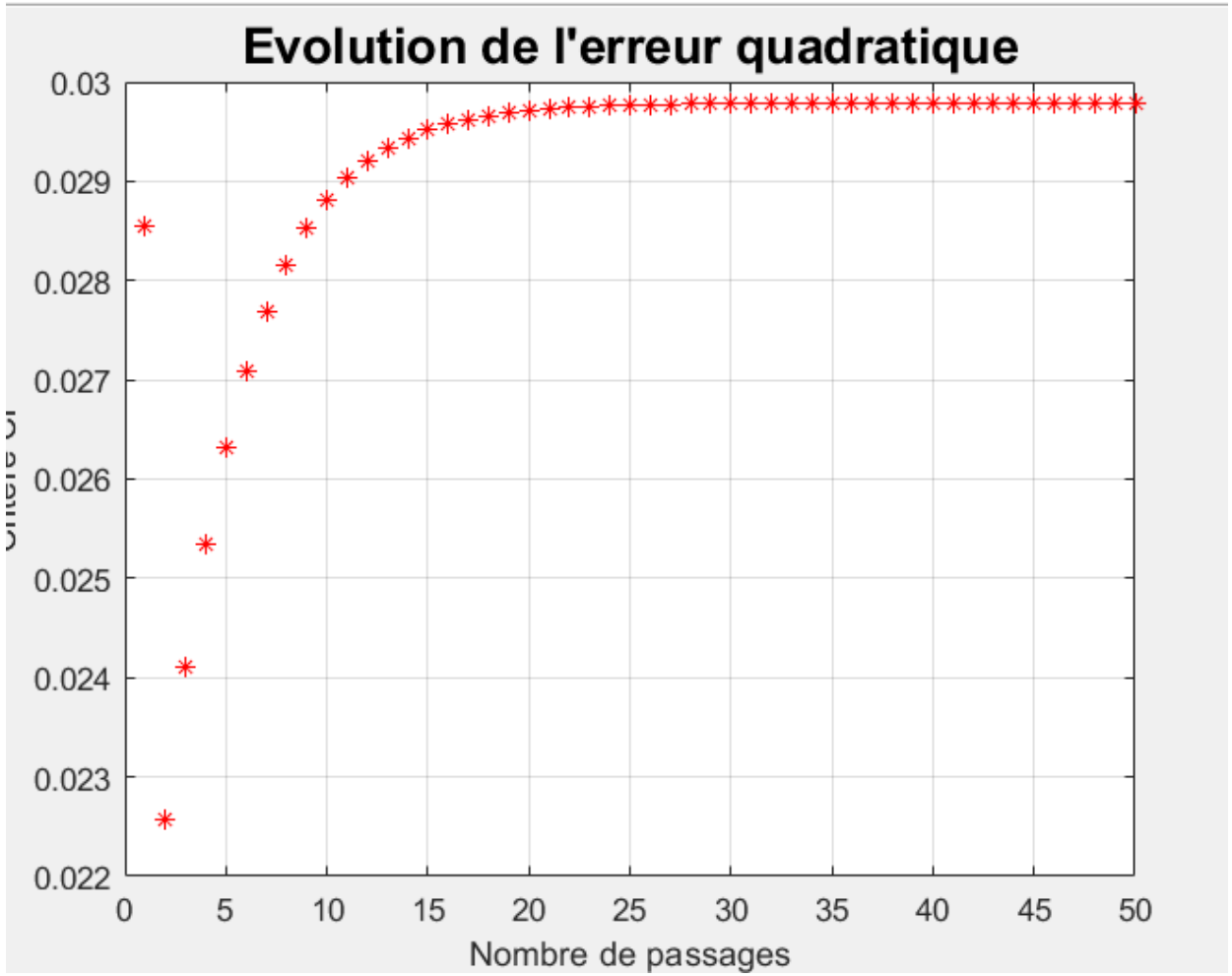


Figure – Evolution du critére d'apprentissage

```
Erreur quadratique de validation = 0.010636
>>
```

Figure – Erreur du test

Réseaux 10 neurones

Code :

```
%% ================ Test ================
Ntest = 100;                    % nombre d'échantillons de test
u_test = 0.5*(rand(1,Ntest)-0.5);  % nouvelle commande (non vue en apprentissage)

y_test(1)=0;
y_test(2)=0;
y1_test(1)=0;
y1_test(2)=0;

%% Réponse réelle du système (validation)
for k = 3:Ntest
    a_test(k)=y_test(k-2);
    y_test(k) = -0.8 * a_test(k) * (u_test(k-1))^2 / (1 + (u_test(k-1))^2) ...
                + 0.033*y_test(k-1) + 0.5*u_test(k-1);
end

%% Réponse du réseau (sans apprentissage)
for k = 3:Ntest
    % vecteur d'entrée
    X_test = [y_test(k-1); u_test(k-1)];

    % propagation avant uniquement
    V1_test = W1*X_test - b1;
    S1_test = tanh(V1_test);
    V2_test = W2*S1_test - b2;
    y1_test(k) = tanh(V2_test);
end
```

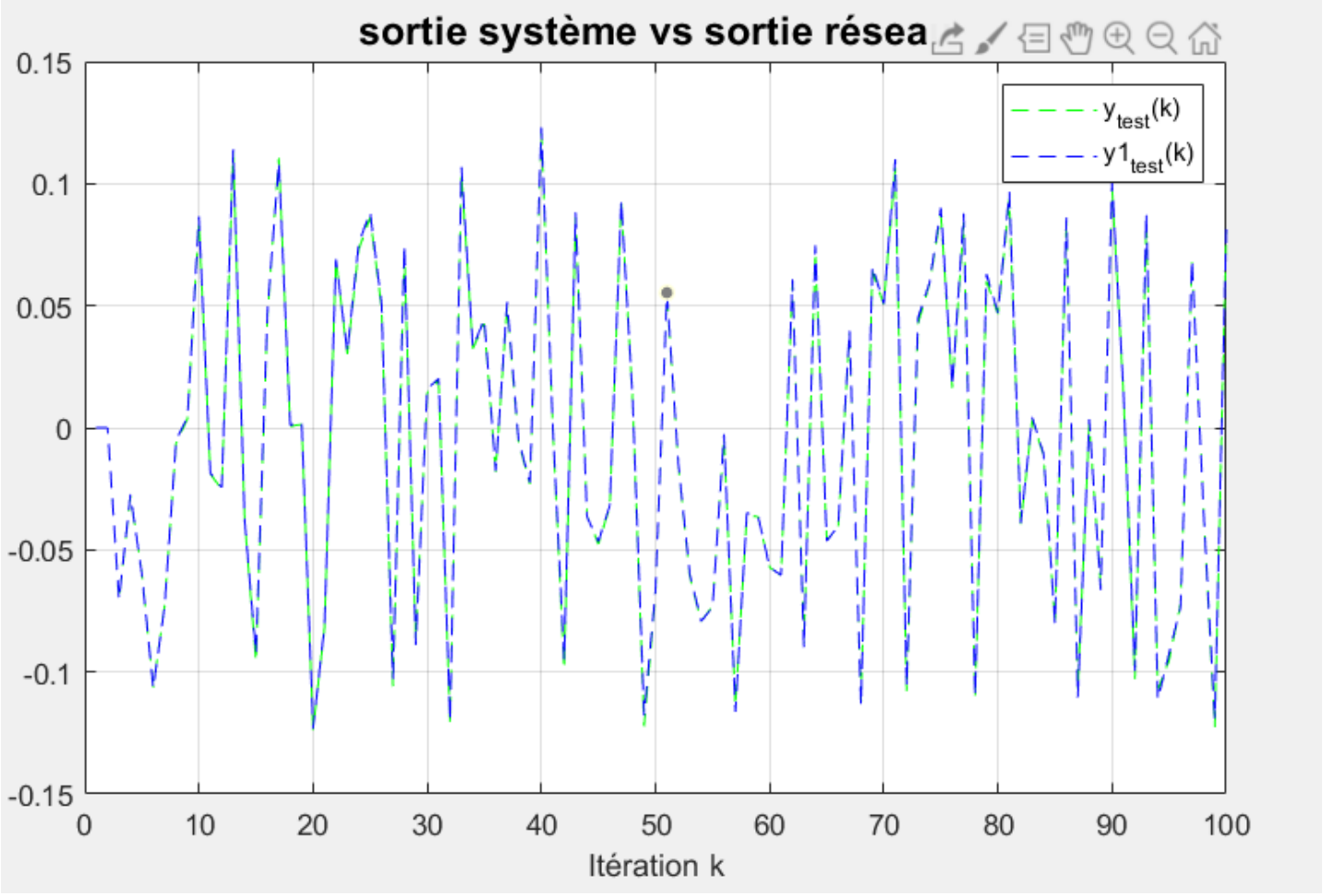


Figure – Sorties du Test

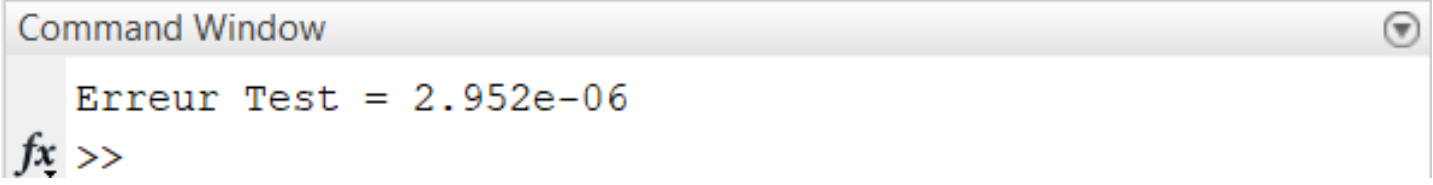


Figure – Erreur de test

5. Conclusion

Résumé des contributions :

- Modélisation d'un système thermique non linéaire
- Conception d'une approche de commande par RNA

Prochaines étapes :

- Implémentation complète
- Validation expérimentale
- Analyse comparative approfondie

Références et Bibliographie


B. Tlili (2008). *Modélisation et commande des systèmes électriques*. Thèse de doctorat, ENSAP. https://theses.hal.science/tel00589735/file/THESE_B_TLILI_ACS_ENIT_2008.pdf


Auteur (1998). *Méthodologie de modélisation et de commande par réseaux de neurones*. Thèse de doctorat, Université de Lille. https://pepite-depot.univ-lille.fr/LIBRE/Th_Num/1998/50376-1998-309.pdf


K. Ben Saad (2020). *Cours d'Automatique générale*, ENIT. https://fr.scribd.com/document/493499963/cours-automatique-generale


J. Haggége (2012). *Cours d'automatique*, ENIT. https://fr.scribd.com/document/311533640/cours-automatique-pdf

M. Ould Moussa (2012). *Modélisation du comportement dynamique non linéaire des structures en MMF*. Thèse de doctorat, École Polytechnique.https://pastel.hal.science/pastel-00675919v1/file/modelisation_du_comportement_dynamique_des_MMFS.pdf

B.Faouzi(2026). *Cours modélisation par réseaux de neurone artificiel*, École nationale d'ingénieurs de Tunis.

Merci de votre attention